STATE MANAGEMENT AND LIFECYCLE

Created: 2025-12-11 Thu 13:45

WHAT WILL BE COVERED

WHAT WILL BE COVERED

- Understanding state management

WHAT WILL BE COVERED

- Understanding state management
- Implementing a state management solution

WHAT WILL BE COVERED

- Understanding state management
- Implementing a state management solution
- Mapping JavaScript events to commands that change the state

WHAT WILL BE COVERED

- Understanding state management
- Implementing a state management solution
- Mapping JavaScript events to commands that change the state
- Updating the state using reducer functions

WHAT WILL BE COVERED

- Understanding state management
- Implementing a state management solution
- Mapping JavaScript events to commands that change the state
- Updating the state using reducer functions
- Re-rendering the view when the state changes

- The state manager keeps the application’s state in sync with the view

- The state manager keeps the application’s state in sync with the view
- Responding to user input by modifying the state accordingly

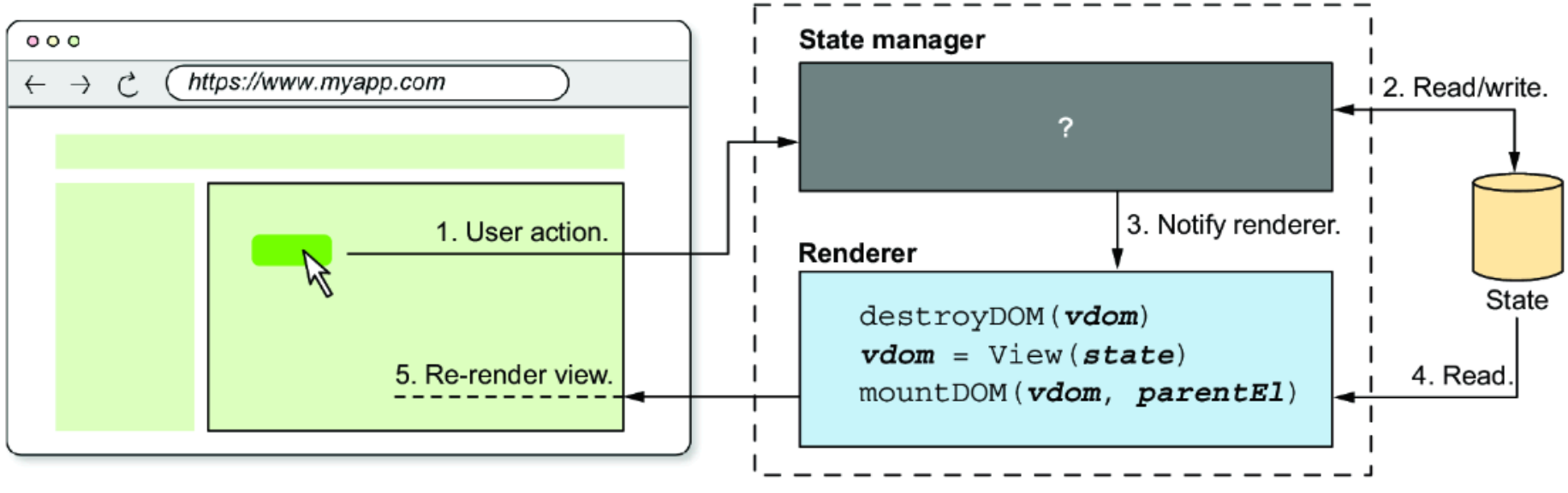
- The state manager keeps the application’s state in sync with the view
- Responding to user input by modifying the state accordingly
- Notifying the renderer when the state has changed.

- The state manager keeps the application's state in sync with the view
- Responding to user input by modifying the state accordingly
- Notifying the renderer when the state has changed.
- The *renderer* is the entity in your framework that takes the virtual Document Object Model (DOM) and mounts it into the browser's document.

- Implement the *renderer* using `mountDOM()` and `destroyDOM()`

- Implement the *renderer* using `mountDOM()` and `destroyDOM()`
- Implement the state manager entities and their communication

- Implement the *renderer* using `mountDOM()` and `destroyDOM()`
- Implement the state manager entities and their communication



- The framework will be rudimentary.

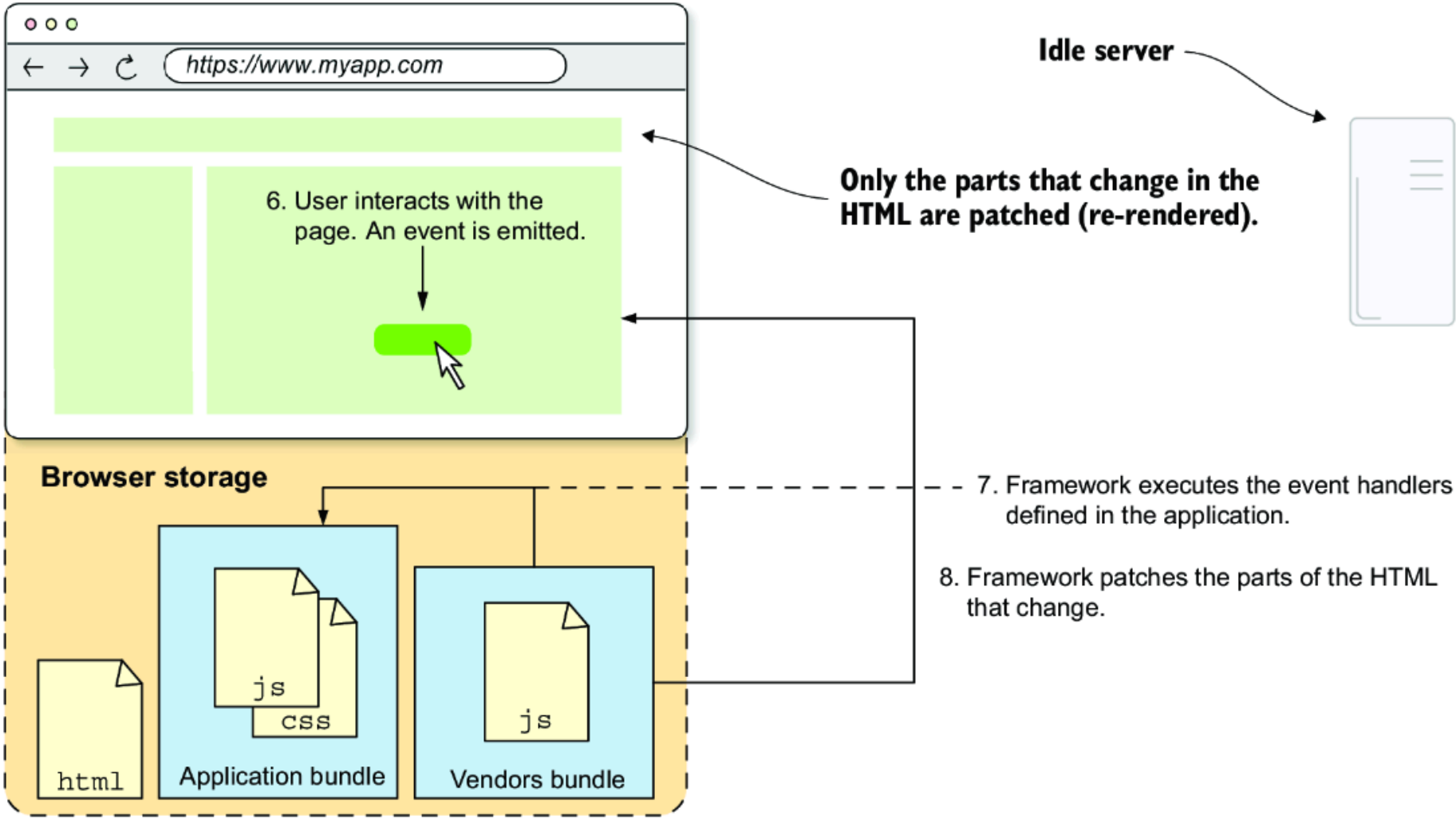
- The framework will be rudimentary.
- The renderer will *destroy* and *mount* the DOM for every state change.

- The framework will be rudimentary.
- The renderer will *destroy* and *mount* the DOM for every state change.
- Three step process:
    1. Destroy the current DOM (calling `destroyDOM()`).
    2. Produce the virtual DOM representing the view given the current state by calling the `View()` function, the top-level component.
    3. Mount the virtual DOM into the real DOM by calling `mountDOM()`.

- The framework will be rudimentary.
- The renderer will *destroy* and *mount* the DOM for every state change.
- Three step process:
    1. Destroy the current DOM (calling `destroyDOM()`).
    2. Produce the virtual DOM representing the view given the current state by calling the `View()` function, the top-level component.
    3. Mount the virtual DOM into the real DOM by calling `mountDOM()`.
- This is not ideal and will further enhance the application but a good starting point.

Focus on handling state changes based on user input

Focus on handling state changes based on user input



WHAT WAS DONE SO FAR

WHAT WAS DONE SO FAR

- `mountDOM()` — takes a virtual DOM tree and a parent DOM element, and mounts the virtual DOM into the parent element.

WHAT WAS DONE SO FAR

- `mountDOM()` — takes a virtual DOM tree and a parent DOM element, and mounts the virtual DOM into the parent element.
  - `createTextNode()` — To create HTML text nodes.

WHAT WAS DONE SO FAR

- `mountDOM()` — takes a virtual DOM tree and a parent DOM element, and mounts the virtual DOM into the parent element.
  - `createTextNode()` — To create HTML text nodes.
  - `createElementNode()` — To create HTML element nodes. This function uses two subfunctions: `addEventListeners()` and `setAttributes()`.

WHAT WAS DONE SO FAR

- `mountDOM()` — takes a virtual DOM tree and a parent DOM element, and mounts the virtual DOM into the parent element.
  - `createTextNode()` — To create HTML text nodes.
  - `createElementNode()` — To create HTML element nodes. This function uses two subfunctions: `addEventListeners()` and `setAttributes()`.
  - `createFragmentNodes()`—To create lists of nodes that have a common parent.

WHAT WAS DONE SO FAR

WHAT WAS DONE SO FAR

WHAT WAS DONE SO FAR

WHAT WAS DONE SO FAR

WHAT WAS DONE SO FAR

THE STATE MANAGER

THE STATE MANAGER

Chronology of user interaction

THE STATE MANAGER

Chronology of user interaction

- *The user* — Interacts with the application's view (e.g. clicks a button).

THE STATE MANAGER

Chronology of user interaction

- *The user* — Interacts with the application's view (e.g. clicks a button).
- *The browser* — Dispatches a native JavaScript event, such as `MouseEvent` or `KeyboardEvent`.

THE STATE MANAGER

Chronology of user interaction

- *The user* — Interacts with the application's view (e.g. clicks a button).
- *The browser* — Dispatches a native JavaScript event, such as `MouseEvent` or `KeyboardEvent`.
- *The application developer* — programmed the framework to know how to update the state for each event.

THE STATE MANAGER

THE STATE MANAGER

- *The framework's state manager* — Updates the state according to the application developer's instructions.

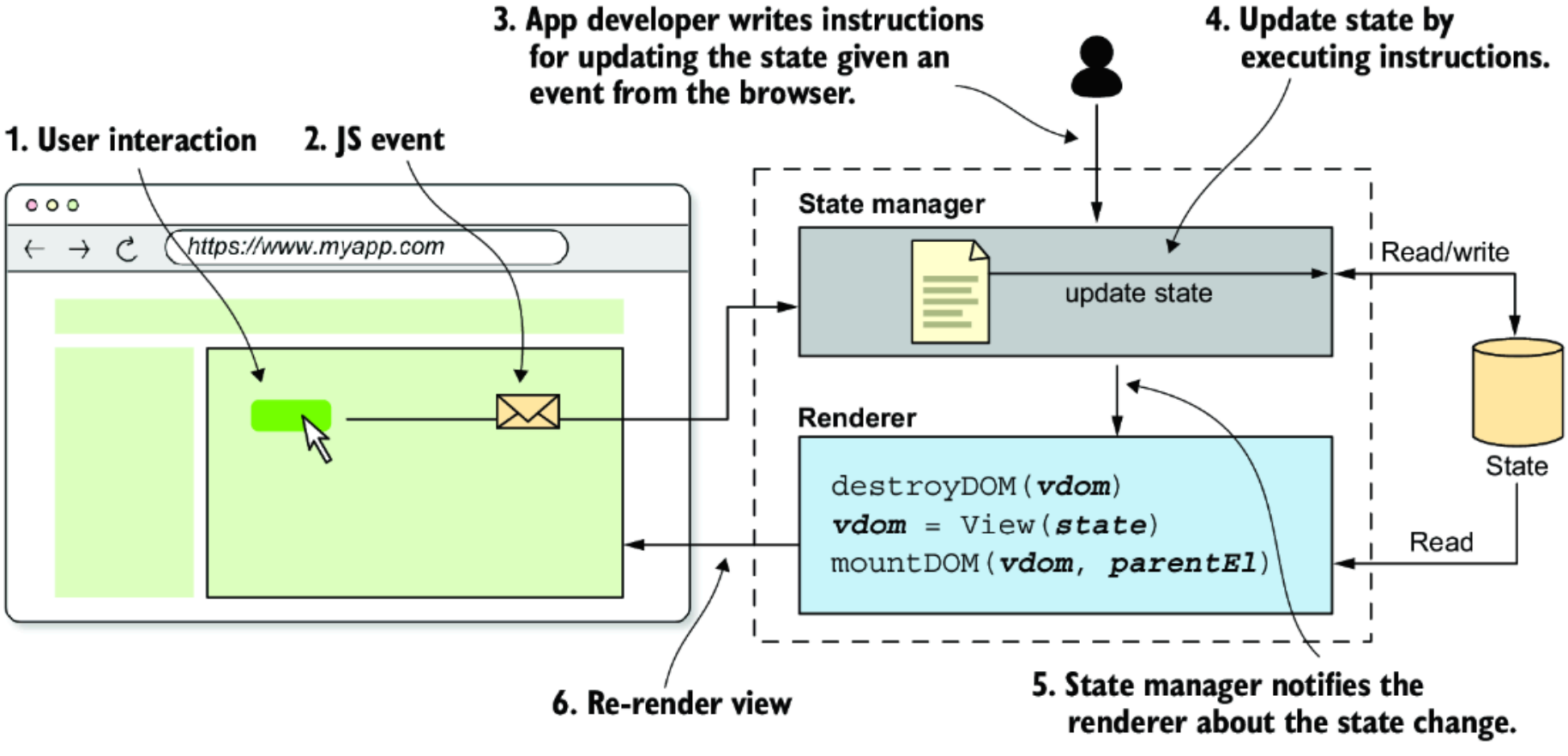
THE STATE MANAGER

- *The framework's state manager* — Updates the state according to the application developer's instructions.
- *The framework's state manager* — Notifies the renderer that the state has changed.

THE STATE MANAGER

- *The framework's state manager* — Updates the state according to the application developer's instructions.
- *The framework's state manager* — Notifies the renderer that the state has changed.
- *The framework's renderer* — Re-renders the view with the new state.

THE STATE MANAGER



THE STATE MANAGER

Two main questions:

THE STATE MANAGER

Two main questions:

- How to update the state when a particular event is dispatched?

THE STATE MANAGER

Two main questions:

- How to update the state when a particular event is dispatched?
- How the state manager will execute the instructions?

MAPPING EVENTS TO DOMAIN COMMANDS

MAPPING EVENTS TO DOMAIN COMMANDS

- JavaScript events don’t have concrete meaning in the application domain.

MAPPING EVENTS TO DOMAIN COMMANDS

- JavaScript events don’t have concrete meaning in the application domain.
- The application developer is the one who *translates* user actions into something meaningful for the application.

MAPPING EVENTS TO DOMAIN COMMANDS

MAPPING EVENTS TO DOMAIN COMMANDS

- In the original **TODOs** application

MAPPING EVENTS TO DOMAIN COMMANDS

- In the original **TODOs** application
  - When the user clicked the *Add* button or pressed the *Enter* key, they wanted to add a new TODO item to the list.

MAPPING EVENTS TO DOMAIN COMMANDS

- In the original **TODOs** application
  - When the user clicked the *Add* button or pressed the *Enter* key, they wanted to add a new TODO item to the list.
  - *"Adding a to-do"* is framed in the language of the application, whereas *"clicking a button"* is a generic thing to do.

MAPPING EVENTS TO DOMAIN COMMANDS

MAPPING EVENTS TO DOMAIN COMMANDS

- Determine what that event means in terms of the application domain.

MAPPING EVENTS TO DOMAIN COMMANDS

- Determine what that event means in terms of the application domain.
- Maps the event to a command that the framework can understand.

MAPPING EVENTS TO DOMAIN COMMANDS

- Determine what that event means in terms of the application domain.
- Maps the event to a command that the framework can understand.
    - A *command* is a request to do something, as opposed to an *event*, which is a notification of something that has happened.

MAPPING EVENTS TO DOMAIN COMMANDS

- Determine what that event means in terms of the application domain.
- Maps the event to a command that the framework can understand.
    - A *command* is a request to do something, as opposed to an *event*, which is a notification of something that has happened.
- These commands ask the framework to update the state; they are expressed in the domain language of the application.

SIDENOTE: EVENT VS. COMMANDS

SIDENOTE: EVENT VS. COMMANDS

- An *event* is a notification of something that has happened.
  - Example: “`A button was clicked`”, "`A network request was completed`”.

SIDENOTE: EVENT VS. COMMANDS

- An *event* is a notification of something that has happened.
  - Example: “`A button was clicked`”, “`A network request was completed`”.
- A *command* is a request to do something in a particular context.
  - Example: `"Add todo"`, `"Edit todo"`,

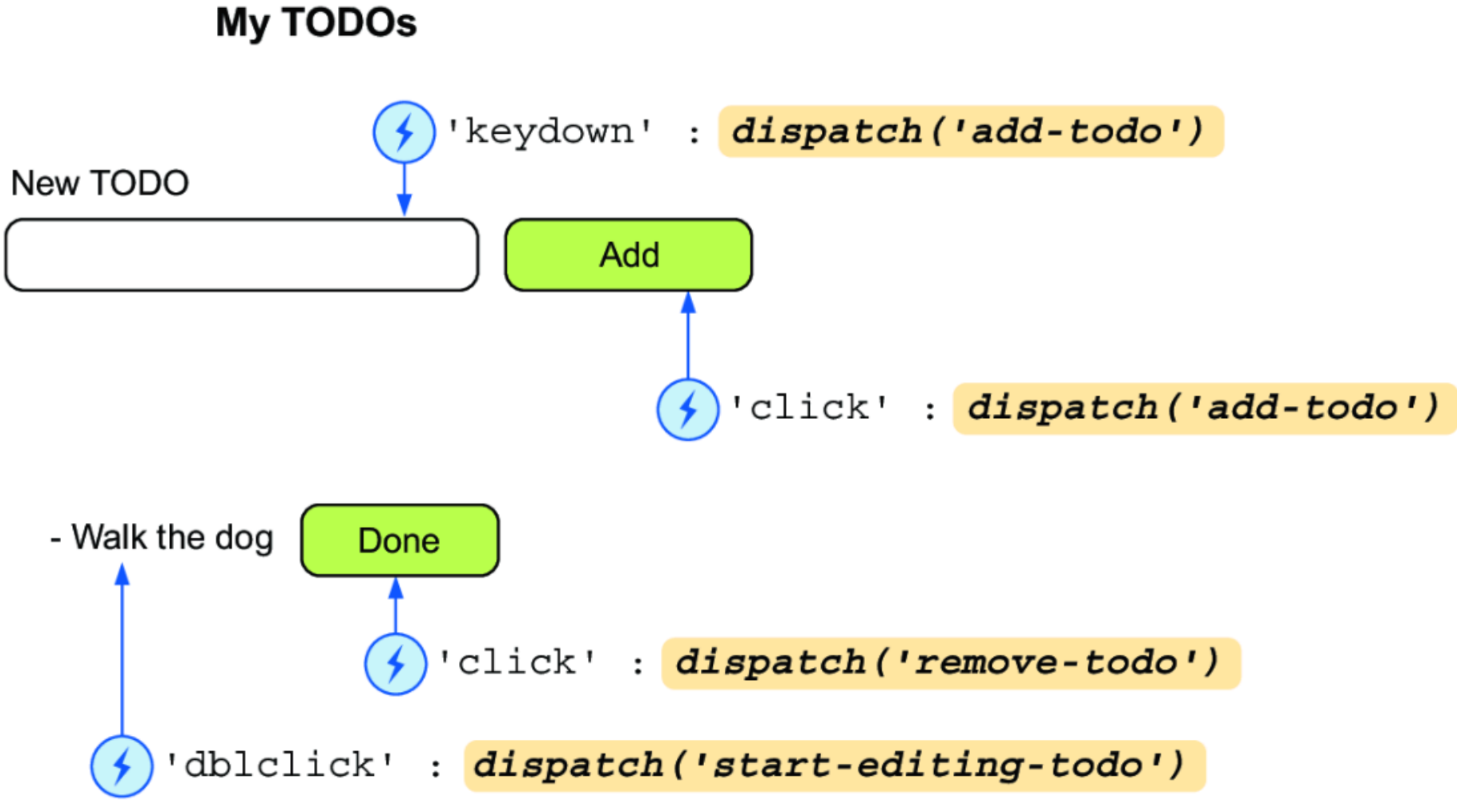
MAPPING TABLE

Browser event	Command	Explanation
Click the Add button.	`add-todo`	Clicking the Add button adds a new to-do item to the list.
Press the `Enter` key (while the input field is focused).	`add-todo`	Pressing the Enter key adds a new to-do item to the list.

MAPPING TABLE

Browser event	Command	Explanation
Click the Done button.	`remove-todo`	Clicking the Done button marks the to-do item as done, removing it from the list.

MAPPING TABLE



REDUCER FUNCTIONS

REDUCER FUNCTIONS

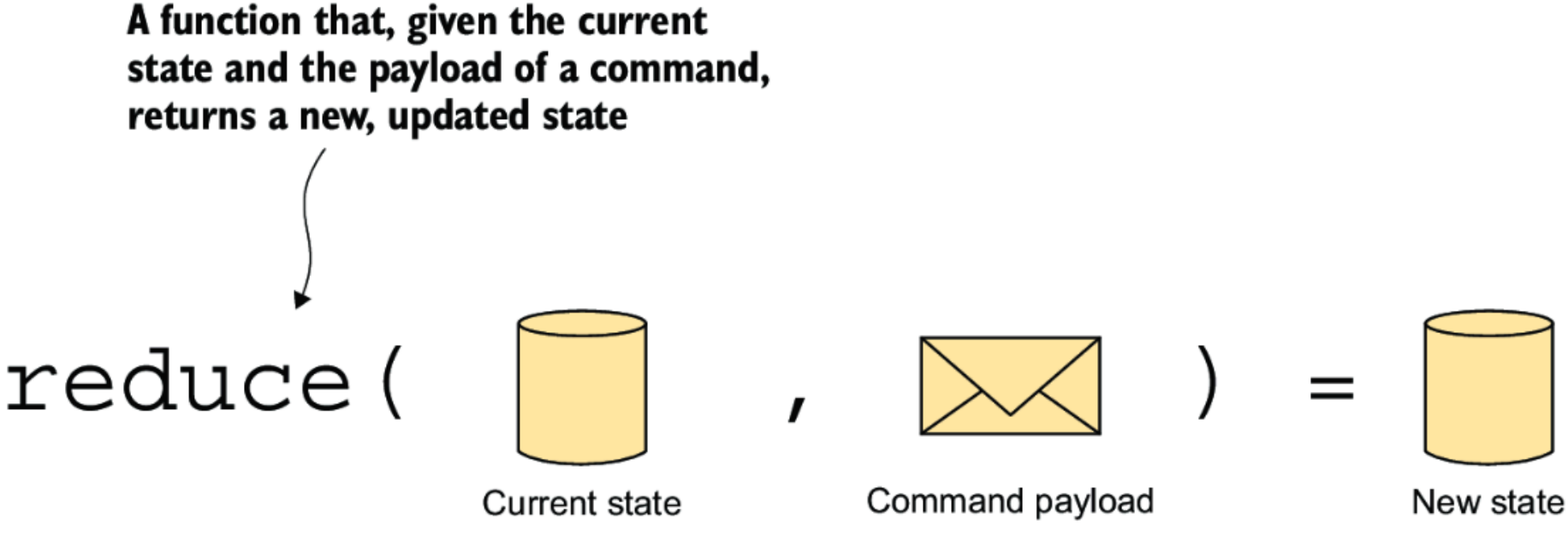
- *Reducer* functions will be implemented by using pure functions and making data immutable

REDUCER FUNCTIONS

- *Reducer* functions will be implemented by using pure functions and making data immutable
- The state is currently immutable, so the functions should create a new state

REDUCER FUNCTIONS

- *Reducer* functions will be implemented by using pure functions and making data immutable
- The state is currently immutable, so the functions should create a new state



REDUCER FUNCTIONS

REDUCER FUNCTIONS

- A *reducer*, in our context, is a function that takes the current state and a payload (the command's data) and returns a new updated state.

REDUCER FUNCTIONS

- A *reducer*, in our context, is a function that takes the current state and a payload (the command's data) and returns a new updated state.
- Never mutate the state that's passed to them (mutation is a side effect)

REDUCER FUNCTIONS

- A *reducer*, in our context, is a function that takes the current state and a payload (the command's data) and returns a new updated state.
- Never mutate the state that's passed to them (mutation is a side effect)
- They create a new state (to remain **pure functions**).

REDUCER FUNCTIONS

- A *reducer*, in our context, is a function that takes the current state and a payload (the command's data) and returns a new updated state.
- Never mutate the state that's passed to them (mutation is a side effect)
- They create a new state (to remain **pure functions**).
- Similar to redux

EXAMPLE

To create a new version of the state when the user removes a to-do item

EXAMPLE

To create a new version of the state when the user removes a to-do item

```
function removeTodo(state, todoIndex) {
  return state.toSpliced(todoIndex, 1)
}
```

EXAMPLE

To create a new version of the state when the user removes a to-do item

```
function removeTodo(state, todoIndex) {
  return state.toSpliced(todoIndex, 1)
}
```

For the state:

EXAMPLE

To create a new version of the state when the user removes a to-do item

```
function removeTodo(state, todoIndex) {
  return state.toSpliced(todoIndex, 1)
}
```

For the state:

```
let todos = ['Walk the dog', 'Water the plants', 'Sand the chairs']
todos = removeTodo(todos, 1)
//todos = ['Walk the dog', 'Sand the chairs']
```

THE DISPATCHER

THE DISPATCHER

- An entity responsible for dispatching the commands to the functions that handle the command.

THE DISPATCHER

- An entity responsible for dispatching the commands to the functions that handle the command.
- the application developer must specify which handler function (or functions) the system should execute in response to each command.

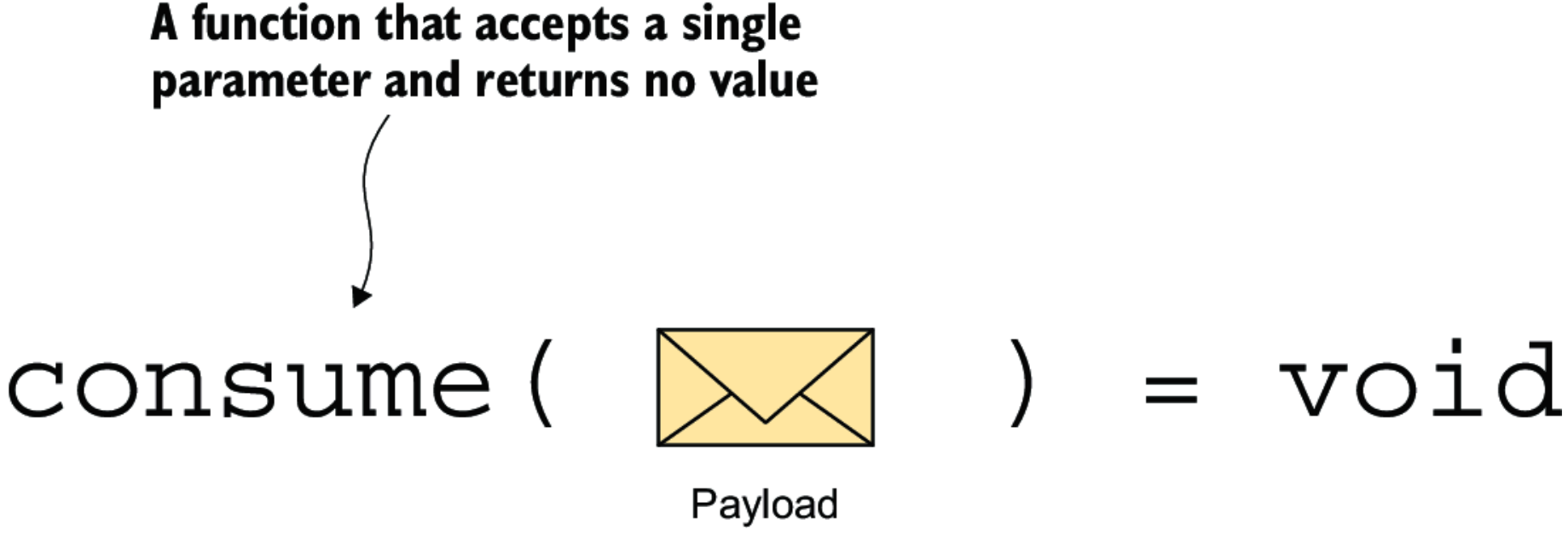
THE CONSUMER

THE CONSUMER

- *Consumer* is a function that accepts a single parameter — the command's payload, in this case — and returns no value.

THE CONSUMER

- *Consumer* is a function that accepts a single parameter — the command's payload, in this case — and returns no value.



THE CONSUMER

```
function removeTodoHandler(todoIndex) {
 // Calls the removeTodo() reducer function to update the state.
  state = removeTodo(state, todoIndex)
}
```

THE CONSUMER

THE CONSUMER

- The command-handler function that removes a to-do from the list receives the to-do index as its single parameter and then calls the `removeTodo()` reducer function to update the state.

THE CONSUMER

- The command-handler function that removes a to-do from the list receives the to-do index as its single parameter and then calls the `removeTodo()` reducer function to update the state.
- It's the dispatcher's responsibility to execute the handler function in response to the `'remove-todo'` command.

THE CONSUMER

- The command-handler function that removes a to-do from the list receives the to-do index as its single parameter and then calls the `removeTodo()` reducer function to update the state.
- It's the dispatcher's responsibility to execute the handler function in response to the `'remove-todo'` command.
- But how do you tell the dispatcher which handler function to execute in response to a command?

ASSIGNING HANDLERS TO COMMANDS

ASSIGNING HANDLERS TO COMMANDS

- The dispatcher needs to have a `subscribe()` method that **registers** a consumer function — the handler — to respond to commands with a given name.

ASSIGNING HANDLERS TO COMMANDS

- The dispatcher needs to have a `subscribe()` method that **registers** a consumer function — the handler — to respond to commands with a given name.
- Also provide mechanism to **unregister** a handler.

ASSIGNING HANDLERS TO COMMANDS

- The dispatcher needs to have a `subscribe()` method that **registers** a consumer function — the handler — to respond to commands with a given name.
- Also provide mechanism to **unregister** a handler.
- The `subscribe()` method should return a function that can be called to unregister the handler.

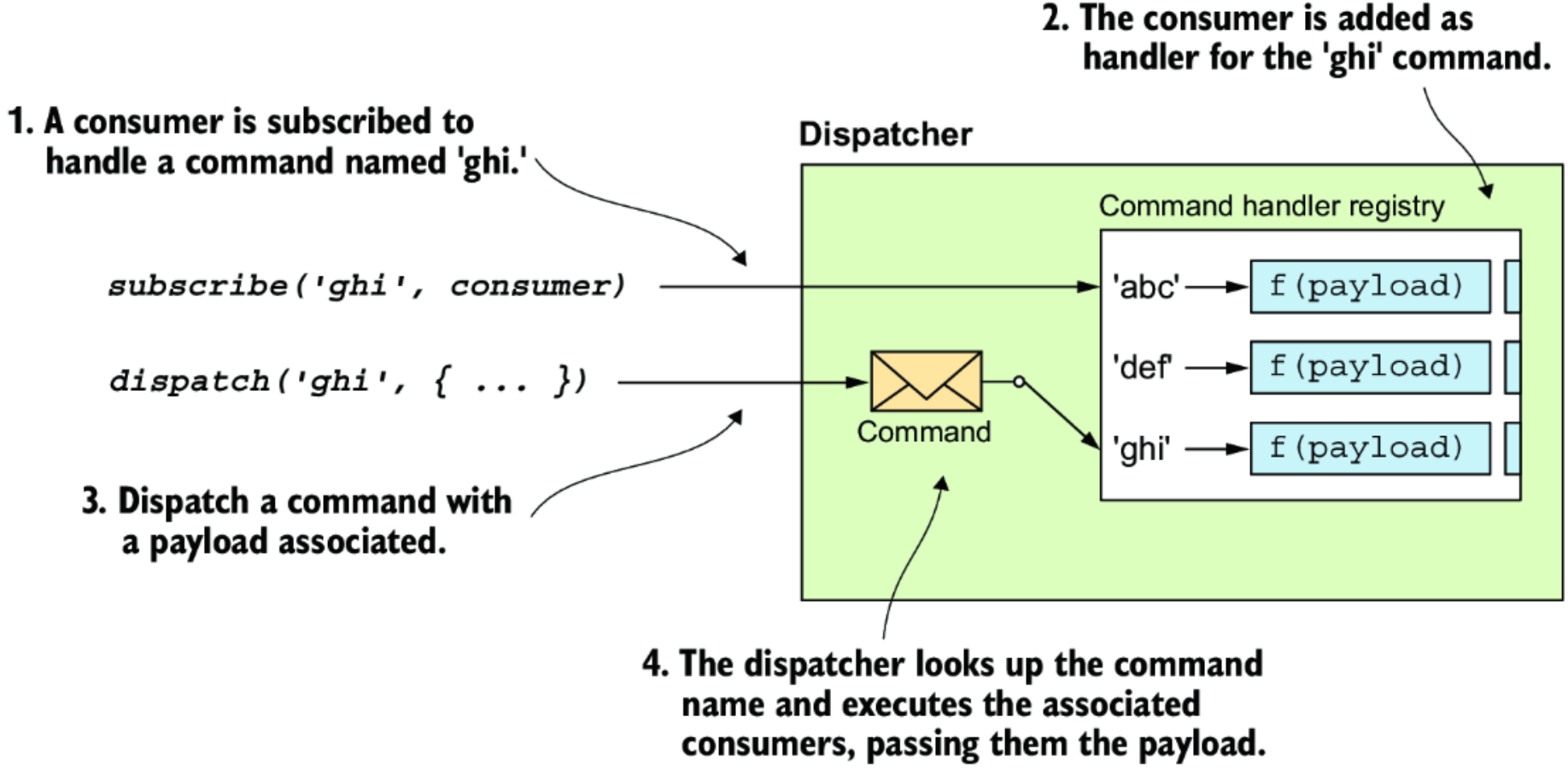
ASSIGNING HANDLERS TO COMMANDS

ASSIGNING HANDLERS TO COMMANDS

- Create a `dispatch()` method that executes the handler functions associated with a command.

ASSIGNING HANDLERS TO COMMANDS

- Create a `dispatch()` method that executes the handler functions associated with a command.



ASSIGNING HANDLERS TO COMMANDS

Create the `dispatcher.js` file inside `src`

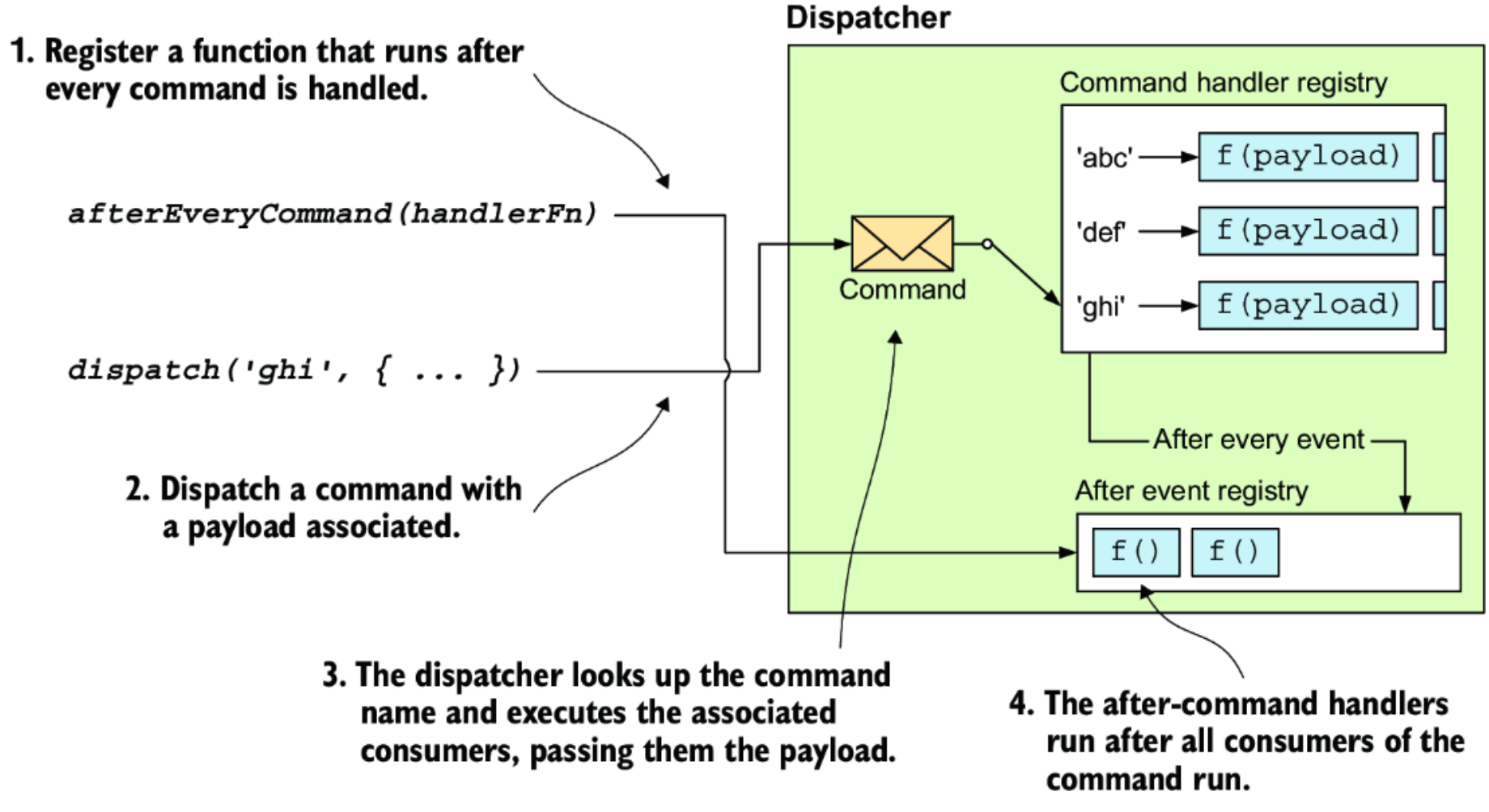
```
src/
├── utils/
│   └── arrays.js
├── attributes.js
├── destroy-dom.js
├── dispatcher.js
├── events.js
├── h.js
├── index.js
└── mount-dom.js
```

```
export class Dispatcher {
  #subs = new Map() //Private variable, defined in ES2020
  subscribe(commandName, handler) {
    if (!this.#subs.has(commandName)) { // Array of subscriptions
      this.#subs.set(commandName, [])
    }

    const handlers = this.#subs.get(commandName)
    if (handlers.includes(handler)) { // Checks if the handler is r
      return () => {}
    }

    handlers.push(handler) // Registers the handler

    return () => {  // Returns function to unregister the handler
      const idx = handlers.indexOf(handler)
      handlers.splice(idx, 1)
    }
  }
}
```

NOTIFYING THE RENDERER ABOUT STATE CHANGES

- The state can change only in response to commands.
- The best time to notify the renderer about state changes is after the handlers for a given command have been executed.
- Allow the dispatcher to register special handler functions (*after-command handlers*), executed after the handlers for any dispatched command have been executed.
- The framework uses these handlers to notify the renderer about potential state changes so that it can update the view.

NOTIFYING THE RENDERER ABOUT STATE CHANGES



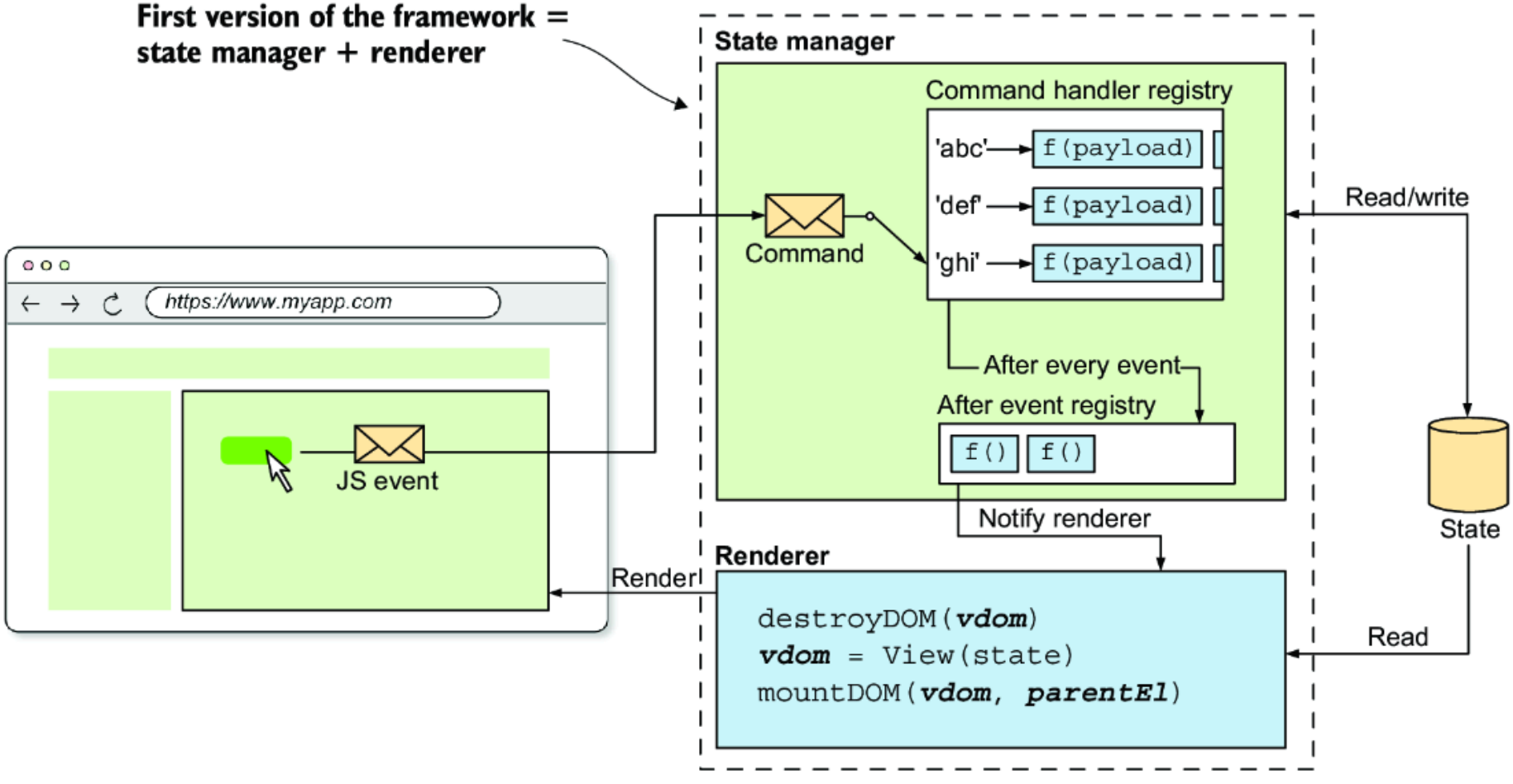
NOTIFYING THE RENDERER ABOUT STATE CHANGES

```
// FILE: dispatcher.js
export class Dispatcher {
  #subs = new Map()
  #afterHandlers = []

 // --snip-- //

  afterEveryCommand(handler) {
    this.#afterHandlers.push(handler) // Registers the handlers

    return () => { // Returns a function to unregister the handler
      const idx = this.#afterHandlers.indexOf(handler)
      this.#afterHandlers.splice(idx, 1)
    }
  }
}
```

DISPATCHING COMMAND

- The `dispatch` method takes two parameters:
  - the name of the command to dispatch
  - and its payload.

```
// FILE: src/dispatcher.js
export class Dispatcher {
  // --snip-- //
  dispatch(commandName, payload) {
    if (this.#subs.has(commandName)) { // Checks registered handler
      this.#subs.get(commandName).forEach((handler) => handler(payl
    } else {
      console.warn(`No handlers for command: ${commandName}`)
    }
    this.#afterHandlers.forEach((handler) => handler()) // Runs the
  }
}
```


First version of the framework =
state manager + renderer
State manager
Command handler registry
'abc'
f(payload)
'def'
f(payload)
'ghi'
f(payload)
Command
Read/write
After every event
After event registry
f()
f()
Notify renderer
State
https://www.myapp.com
JS event
Render
Renderer
destroyDOM(vdom)
vdom = View(state)
mountDOM(vdom, parentEl)
Read

dispatcher.js

```
export class Dispatcher {
  #subs = new Map()
  #afterHandlers = []

  subscribe(commandName, handler) {
    if (!this.#subs.has(commandName)) {
      this.#subs.set(commandName, [])
    }

    const handlers = this.#subs.get(commandName)
    if (handlers.includes(handler)) {
      return () => {}
    }

    handlers.push(handler)

    return () => {
      const idx = handlers.indexOf(handler)
      handlers.splice(idx, 1)
    }
  }
```

dispatcher.js

```
  afterEveryCommand(handler) {
    this.#afterHandlers.push(handler)

    return () => {
      const idx = this.#afterHandlers.indexOf(handler)
      this.#afterHandlers.splice(idx, 1)
    }
  }

  dispatch(commandName, payload) {
    if (this.#subs.has(commandName)) {
      this.#subs.get(commandName).forEach((handler) => handler(payl
    } else {
      console.warn(`No handlers for command: ${commandName}`)
    }

    this.#afterHandlers.forEach((handler) => handler())
  }
}
```

ASSEMBLING THE STATE MANAGER

APPLICATION INSTANCE

APPLICATION INSTANCE

- The *application instance* is the object that manages the lifecycle of the application.

APPLICATION INSTANCE

- The *application instance* is the object that manages the lifecycle of the application.
- It manages the state, renders the views, and updates the state in response to user input.

APPLICATION INSTANCE

- The *application instance* is the object that manages the lifecycle of the application.
- It manages the state, renders the views, and updates the state in response to user input.
- Developers need to pass three things to pass to the application instance:
  - The initial state of the application
  - The reducers that update the state in response to commands
  - The top-level component of the application

APPLICATION INSTANCE

- The framework will take care of:
  - instantiating a renderer
  - state manager
  - wiring them together.
- The application instance can expose a `mount()` method that takes a DOM element as a parameter, mounts the application in it, and kicks off the application's lifecycle.

THE APPLICATION INSTANCE'S RENDERER

THE APPLICATION INSTANCE'S RENDERER

THE APPLICATION INSTANCE'S RENDERER

THE APPLICATION INSTANCE'S RENDERER

- Two variables in the closure of the `createApp()` function:
  - `parentEl` - keep track of the DOM element
  - `vdom` - the virtual DOM tree of the previous view.
  - Both should be initialized to `null`

- `renderApp()` - function that renders the view by destroying the current DOM tree (if one exists) and then mounting the new one.

```
// FILE: src/app.js
import { destroyDOM } from './destroy-dom'
import { mountDOM } from './mount-dom'

export function createApp({ state, view }) {
  let parentEl = null
  let vdom = null

  function renderApp() {
    if (vdom) {
      destroyDOM(vdom) // If previous view exists, unmount it
    }

    vdom = view(state)
    mountDOM(vdom, parentEl) // Mounts the new view
  }

  return {
    mount(_parentEl) { // Method to mount the application to the DOM
      parentEl = _parentEl
      renderApp()
    },
  }
}
```

THE APPLICATION INSTANCE'S STATE MANAGER

- The state manager is more complex
  - use the `Dispatcher` class
  - wrap the *state reducers* in a *consumer function* that the dispatcher will call everytime a command is dispatched. Let's see how this is done.

```
import { destroyDOM } from './destroy-dom'
import { Dispatcher } from './dispatcher'
import { mountDOM } from './mount-dom'

export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

  const dispatcher = new Dispatcher()
  // Re-renders the app after every command
  const subscriptions = [dispatcher.afterEveryCommand(renderApp)]

  for (const actionName in reducers) {
    const reducer = reducers[actionName]

    const subs = dispatcher.subscribe(actionName, (payload) => {
      // Updates the state calling the reducer function
      state = reducer(state, payload)
    })
    // Adds each command subscription to the subscriptions array
    subscriptions.push(subs)
  }
 // --snip-- //
}
```

THE APPLICATION INSTANCE'S STATE MANAGER

THE APPLICATION INSTANCE'S STATE MANAGER

- The `afterEveryCommand()` function returns a function that unsubscribes the handler
- Iterate the `reducers` object
  - wrap each of the reducers inside a handler function that calls the reducer and updates the state,
  - Subscribe the handler to the dispatcher.

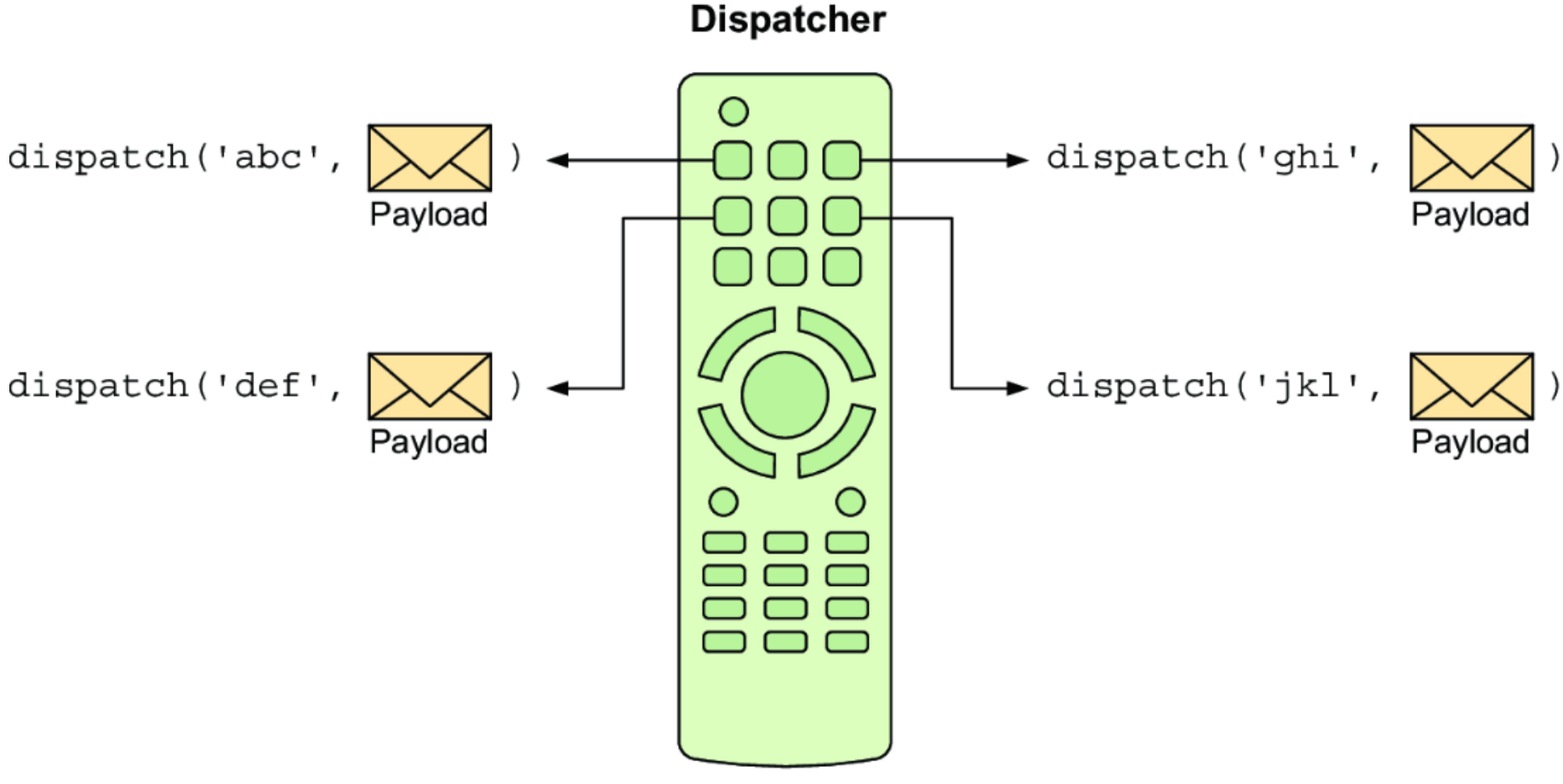
COMPONENTS DISPATCHING COMMANDS

- *VDOM* allows attaching event listeners to DOM elements

```
h( 'button',
   { on: { click: () => { ... } } },
   ['Click me']
)
```

COMPONENTS DISPATCHING COMMANDS

- Need to pass the dispatcher to the components



COMPONENTS DISPATCHING COMMANDS

The component can dispatch the commands to the `dispatch` method.

```
h(
  'button',
  {
    on: {
      click: () => dispatcher.dispatch('remove-todo', todoIdx)
    }
  },
  ['Done']
)
```

COMPONENTS DISPATCHING COMMANDS

```
// FILE: src/app.js
export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

  const dispatcher = new Dispatcher()
  const subscriptions = [dispatcher.afterEveryCommand(renderApp)]

  function emit(eventName, payload) {
    dispatcher.dispatch(eventName, payload)
  }
 // --snip-- //
  function renderApp() {
    if (vdom) {
      destroyDOM(vdom)
    }

    vdom = view(state, emit)
    mountDOM(vdom, parentEl)
  }
 // --snip-- //
}
```

COMPONENTS DISPATCHING COMMANDS

- Implement an `emit()` function.
  - Pass the `emit()` function to the components as a second argument.
- Onwards, the components will receive two arguments:
  - the state
  - the `emit()` function.
  - If a component doesn't need to dispatch commands, it can ignore the second argument.

UNMOUNTING THE APPLICATION

- When an application instance is created:
  - the state reducers are subscribed to the dispatcher
  - the `renderApp()` function is subscribed to the dispatcher as an after-command handler.
- When unmounting:
  - destroy the view
  - unsubscribe the reducers
  - unsubscribe the `renderApp()` function from the dispatcher.
- Add an `unmount()` method to the app instance

UNMOUNTING THE APPLICATION

```
export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

 // --snip-- //

  return {
    mount(_parentEl) {
      parentEl = _parentEl
      renderApp()
    },

    unmount() {
      destroyDOM(vdom)
      vdom = null
      subscriptions.forEach((unsubscribe) => unsubscribe())
    },
  }
}
```

UNMOUNTING THE APPLICATION

- The `unmount()` method uses the `destroyDOM()` function
- Sets the `vdom` property to `null`
- unsubscribes the reducers and the `renderApp()` function from the dispatcher.

THE COMPLETE `app.js` IMPLEMENTATION

```
import { destroyDOM } from './destroy-dom'
import { Dispatcher } from './dispatcher'
import { mountDOM } from './mount-dom'

export function createApp({ state, view, reducers = {} }) {
  let parentEl = null
  let vdom = null

  const dispatcher = new Dispatcher()
  const subscriptions = [dispatcher.afterEveryCommand(renderApp)]

  function emit(eventName, payload) {
    dispatcher.dispatch(eventName, payload)
  }

  for (const actionName in reducers) {
    const reducer = reducers[actionName]
```

```
    const subs = dispatcher.subscribe(actionName, (payload) => {
      state = reducer(state, payload)
    })
    subscriptions.push(subs)
  }
  function renderApp() {
    if (vdom) {
      destroyDOM(vdom)
    }

    vdom = view(state, emit)
    mountDOM(vdom, parentEl)
  }
```

```
  return {
    mount(_parentEl) {
      parentEl = _parentEl
      renderApp()
    },

    unmount() {
      destroyDOM(vdom)
      vdom = null
      subscriptions.forEach((unsubscribe) => unsubscribe())
    },
  }
}
```

SUMMARY

- Your framework's first version is made of a renderer and a state manager wired together.
- The renderer first destroys the DOM (if it exists) and then creates it from scratch. This process isn't very efficient and creates problems with the focus of input fields, among other things.
- The state manager is in charge of keeping the state and view of the application in sync.

SUMMARY

- The developer of an application maps the user interactions to commands, framed in the business language, that are dispatched to the state manager.
- The commands are processed by the state manager, updating the state and notifying the renderer that the DOM needs to be updated.
- The state manager uses a reducer function to derive the new state from the old state and the command’s payload.